# Webhooks & Results Retrieval

How to have completed verification data - JSON and PDF - delivered into your own systems instead of reviewing it in the Truv dashboard. What it requires, who can build it, and how to prove it works.

## FIRST: DO YOU ACTUALLY NEED THIS?

Webhooks are optional. Verification results are always available in the Truv dashboard without building anything. Choose deliberately, because option B means you now host an internet-facing endpoint that receives personal data.

| OPTION | HOW RESULTS REACH YOU | WHAT YOU BUILD | TRADE-OFF |
|---|---|---|---|
| **A. Dashboard** | Staff open the Truv dashboard and review completed verifications. | Nothing. | **No verification data at rest in your environment.** Nothing to secure, nothing to audit, no code. |
| **B. Webhook** *(this guide)* | Truv POSTs to your URL the moment an order finishes; you then fetch the data. | An HTTPS endpoint, built by a developer. | Results flow into your system automatically, in near real time. You take on custody of the data and the endpoint. |
| **C. Polling** | You call Truv on a timer and ask whether each order is done. | A scheduled job. | No inbound endpoint at all - the right answer when a public endpoint is not permitted. Slower and wasteful, but perfectly workable. |

## WHO CAN SET THIS UP?

By tier. For results delivered into your systems, Tier 2 is the recommended build.

| TIER | APPROACH | WHO CAN DO IT | WHAT IT TAKES | VERDICT |
|---|---|---|---|---|
| **0** | No webhook. Dashboard review. | Anyone. No technical skill. | None | Genuinely fine. Do not build anything you do not need. |
| **1** | No-code receiver (Zapier, Make, Power Automate). | A business analyst. | No code | **Not recommended for real data.** These tools generally cannot verify the signature over the raw request body, which is the one non-negotiable security step. Acceptable only for "an order finished" notifications that carry no verification data. |
| **2** | A real HTTPS endpoint in whatever stack you already run. | **Any backend developer.** No Truv-specific knowledge needed. | A small service | **The right answer.** It is four small behaviors, all shown in this guide. A working reference receiver ships with the intake kit. |
| **3** | Tier 2 plus mTLS, OAuth 2.0, or custom-header auth, behind an API gateway. | A platform or security team, coordinating with Truv. | Gateway and security review | Only when your security policy demands it. Test the gateway with Truv's sandbox early - a gateway that rejects Truv's request fails before your application ever sees it. |

> **Scope check for a Tier 2 build.** Your endpoint has exactly four jobs: verify the signature, answer `200` fast, ignore anything it has already seen, and fetch the order when it is finalized. That is the entire contract.

## HOW IT FITS TOGETHER

**01 REGISTER**
Add your HTTPS URL once, per environment. Dashboard or API.

**02 EVENT**
An order finishes. Truv POSTs a small JSON event - **not** the results.

**03 VERIFY**
You check **X-WEBHOOK-SIGN** against the raw body, then reply **200** within 10s.

**04 FETCH**
You call **GET /v1/orders/{id}/** for the JSON and the PDF links.

**05 STORE**
You write it into your own system of record.

**The webhook is a doorbell, not a delivery.** Events carry identifiers and status - never the verification data itself. You always fetch the actual results from the API afterwards. This is deliberate: it means a webhook intercepted in transit exposes nothing,

and it means the API is always the authoritative source of state.

### STEP 1 - REGISTER YOUR ENDPOINT

Configure endpoints **separately for sandbox and production**. In the Truv Dashboard: **Development → Webhooks**, enter your URL, save.

Or over the API:

```
curl -X POST https://prod.truv.com/v1/webhooks/ \
  -H "X-Access-Client-Id: <client_id>" \
  -H "X-Access-Secret: <access_key>" \
  -H "Content-Type: application/json" \
  -d '{
        "name": "Case management intake",
        "webhook_url": "https://your.host/webhooks/truv",
        "env_type": "prod",
        "events": ["order-finalized"]
      }'
```

Full management is available: `GET /v1/webhooks/`, `GET` / `PATCH` / `DELETE /v1/webhooks/{id}/`. If you have the intake kit, `python3 truv_intake.py webhook add https://your.host/webhooks/truv` does this for you and refuses non-HTTPS URLs.

> **Plan for one endpoint per environment.** Register a single receiver per environment and fan out to your internal consumers on your own side after receipt - that is the pattern to design for. If you believe you need several endpoints on one environment, confirm it with Truv before building around it.

### STEP 2 - CHOOSE YOUR EVENTS

Twenty event types exist. For an intake integration you need one, occasionally a few more. Subscribing to everything creates noise you will then have to filter.

| EVENT | SUBSCRIBE? | WHAT IT MEANS |
|---|---|---|
| `order-finalized` | **Yes - this is the one** | The order group has reached its terminal, final state. No further data collection or refresh will occur. **This is your signal to fetch results.** |
| `order-batch-completed` | If you send batches | A batch job finished. Carries `batch_id` and the counts (`order_count`, `created_count`, `failed_count`) - not the orders. Fetch `GET /v1/orders/batch/{batch_id}/` for per-order results. Note that `created_count` includes orders that already *existed*. The intake kit polls the job itself, so it does not need this event. |
| `task-status-updated` | Useful | Fires on individual connection attempts, including failures. Watch for `login_error`, `mfa_error`, `config_error` if you want to help applicants who get stuck, and `done` when data finishes downloading. |
| `certification-completed` | If self-certification is on | The applicant submitted their income self-certification. |
| `order-created` | Rarely | Confirms an order exists. You already know - you created it. `status` is always `pending`. |
| `order-status-updated` | **Not for completion** | Fires **per suborder** - once per employer or financial account, not once per order. A multi-employer order fires it several times and none of them means the order is done. See the warning below. |
| `order-refresh-failed` | If you use refresh | A refresh task failed for an order. |

> **The single most common integration bug.** Teams wire completion logic to `order-status-updated` with `status == "completed"`, because it reads like "the order completed". It is not. It is a *suborder* status - one employer or one financial account. On a two-employer order you will process the applicant twice, and on a failed second employer you will report success too early. Use `order-finalized`. If you must derive completion from the API instead, note that **an Order has no top-level status field** - check `completed_at` / `canceled_at` / `is_expired`.

### STEP 3 - VERIFY THE SIGNATURE. THIS IS NOT OPTIONAL.

Every request carries `X-WEBHOOK-SIGN`: an HMAC-SHA256 of the **raw request body**, keyed with your **Access Secret**, prefixed `v1=`. Without this check, anyone who learns your URL can post fabricated events to it.

```
import hashlib, hmac

def verify(raw_body: bytes, signature_header: str, access_secret: str) -> bool:
    expected = "v1=" + hmac.new(
        key=access_secret.encode("utf-8"),
        msg=raw_body,                       # RAW BYTES - never re-serialized JSON
        digestmod=hashlib.sha256,
    ).hexdigest()
    return hmac.compare_digest(expected, signature_header)
```

**Use the raw bytes.** If your framework parses the JSON and you re-serialize it to verify, the byte representation changes - key order, whitespace, unicode escaping - and every signature check fails. In Express you must capture `rawBody` in the `verify` callback; in Flask use `request.get_data()`, not `request.json`. This costs teams more debugging time than anything else in the integration.

Two smaller points that matter: use your **Access Secret**, not the Client ID. And compare with a constant-time function (`hmac.compare_digest` or equivalent) rather than `==`.

### STEP 4 - ANSWER FAST, THEN DO THE WORK

| RULE | DETAIL |
|---|---|
| **Timeout** | You must return a `2xx` within **10 seconds**. |
| **Retries** | `4xx` and `5xx` trigger up to **3 retries at 30-second intervals**. After that the delivery is marked failed and is not retried again. |
| **Redirects** | `3xx` responses are followed. |
| **Duplicates** | An event may be delivered more than once. Deduplicate on `webhook_id`. |
| **Ordering** | Sequence by the `event_created_at` field, not by arrival order. |
| **Auto-disable** | If your endpoint returns no 2xx/3xx response for a full day, the account owner is emailed. If it is still failing three days after that email, the endpoint is disabled and events sent while it is disabled are not replayed. Re-enable with `PATCH /v1/webhooks/{id}/` and `{"enabled": true}`. |

Fetching an order and downloading a PDF can exceed ten seconds. So acknowledge first and process afterwards - queue the event, or reply then continue on a background thread:

```
@app.post("/webhooks/truv")
def truv_webhook():
    raw = request.get_data()                          # raw bytes
    if not verify(raw, request.headers.get("X-WEBHOOK-SIGN", ""), SECRET):
        return "invalid signature", 401

    event = json.loads(raw)
    if already_seen(event["webhook_id"]):             # idempotency
        return "ok", 200

    queue.enqueue(handle_event, event)                # durable queue, not memory
    return "ok", 200                                  # ...and answer now
# handle_event marks webhook_id as seen only AFTER the fetch succeeds,
# so a failure is retried instead of silently lost.
```

### STEP 5 - FETCH THE RESULTS

On `order-finalized`, take the `order_id` from the event and fetch the order. The response is the verification JSON, and it carries the PDF links.

```
curl https://prod.truv.com/v1/orders/{order_id}/ \
  -H "X-Access-Client-Id: <client_id>" \
  -H "X-Access-Secret: <access_key>"
```

| WHAT YOU WANT | WHERE IT IS |
|---|---|
| **The verification JSON** | The `GET /v1/orders/{id}/` response body itself. |
| **The PDF report** | `pdf_report` on each suborder - under `employers[]` and `financial_accounts[]`. One per connection, so a multi-employer order has several. Authenticate the download with the same two headers. |
| **The assets report** | One consolidated report for the whole order - *not* on any suborder, so a loop over `employers[]` and `financial_accounts[]` never finds it. Take `user_id` and top-level `voa_report_id` from the order, then `GET /v1/users/{user_id}/assets/reports/{voa_report_id}/?fmt=pdf`. Omit `?fmt=pdf` for JSON. If you verify assets, collect this on every order. |
| **Other reports by ID** | `voie_report_id`: `GET /v1/users/{user_id}/reports/{report_id}/`. `income_insights_report_id`: `GET /v1/users/{user_id}/income_insights/reports/{report_id}/`. Both accept `?fmt=pdf`. |
| **Whether it is really done** | `completed_at` / `canceled_at` / `is_expired`. There is no top-level `status`. |

> **Do not promise `voie_report_id` by default.** It is only populated automatically when an account-level setting is enabled - the one that generates a Fannie Mae DU / Freddie Mac LPA compatible report ID on order completion. Ask Truv support to switch it on for your account before you build against it.

### TESTING BEFORE YOU GO LIVE

1. **Register a sandbox endpoint** and get it reachable. For local development, `ngrok http 8080` gives you a public HTTPS URL that forwards to your machine.
2. **Trigger real events** by running a verification through Truv Bridge in the Dashboard Emulator in sandbox.
3. **Confirm Truv reached you** with `GET /v1/webhook-requests/` - a paginated history of the last 30 days of delivery attempts, with the status code your endpoint returned and any error message. This one call answers "did Truv even try" and settles most disputes instantly.
4. **Test the failure paths deliberately:** post a body with a broken signature and confirm you reject it; deliver the same `webhook_id` twice and confirm you process it once.
5. **Only then** register the production endpoint.

A complete runnable receiver - signature verification, fast acknowledgement, idempotency, and fetch-on-finalize, in about 250 lines of standard-library Python - ships with the intake kit as `webhook_receiver_example.py`.

### NETWORK AND HARDENING

- **Allowlist these source IPs** if your network filters inbound traffic: `34.212.57.93`, `44.224.243.166`, `52.25.14.79`.
- Requests arrive with `User-Agent: Truv-Webhook-Service/2.0`. Treat it as a hint, never as authentication.
- **Beyond signatures, if your policy requires it:** mTLS with either Truv-signed or client-signed certificates; OAuth 2.0, where Truv obtains access tokens before delivering; or custom headers such as a client ID and secret. All of these are configured *with* Truv rather than self-serve, and are set per environment - start that conversation early, as it is a scheduling problem more than a technical one.

> **If you put an API gateway in front of your endpoint** - AWS API Gateway, Azure APIM, an authenticating reverse proxy - make sure it does not reject Truv before your application ever sees the request, and that it does not alter the request body. A gateway that rewrites or re-encodes the body will break signature verification even though your code is correct. This is the most common cause of "we registered the webhook and nothing arrived."

## GO-LIVE CHECKLIST

- Endpoint is HTTPS and publicly reachable.
- Signature verified on **every** request, using the raw body and the Access Secret, with a constant-time comparison.
- Returns `2xx` in under 10 seconds; slow work happens after the response.
- Deduplicates on `webhook_id`, in durable storage rather than process memory.
- Subscribed to `order-finalized` - and completion logic is **not** keyed on `order-status-updated`.
- Fetches results from the API rather than trusting the event payload.
- Separate endpoints registered for sandbox and production.
- Failure paths tested: bad signature rejected, duplicate delivery processed once.
- Someone owns the alert when `GET /v1/webhook-requests/` starts showing non-2xx.
- Access Secret stored in a secrets manager, not in source control.